

WFLOW-06: Custom Notification Templates

Series: WFLOW — Workflows and Alert Notifications | **Notebook:** 6 of 10 | **Created:** January 2026 | **Last Updated:** 10/05/2026

Rich Message Formatting

Create professional, informative notifications with dynamic content, formatting, and data enrichment. This notebook covers Jinja templating, Slack Block Kit, Teams Adaptive Cards, and data enrichment patterns.

Table of Contents

- 1. [Template Design Principles](#)
- 2. [Jinja2 Expression Deep Dive](#)
- 3. [Slack Block Kit Templates](#)
- 4. [Teams Adaptive Card Templates](#)
- 5. [Data Enrichment](#)
- 6. [Template Library](#)
- 7. [Testing Templates](#)

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Platform subscription

Requirement	Details
Permissions	<code>automation:workflows:write</code>
Prior Knowledge	WFLOW-01 through WFLOW-05

1. Template Design Principles

Effective Alert Messages

Principle	Good	Bad
Scannable	Key info in first line	Important details buried
Actionable	Include link to problem	"Check Dynatrace"
Contextual	Show affected service	Generic "problem detected"
Severity-coded	Visual severity indicator	Plain text only
Concise	Essential details only	Long descriptions

Message Structure

```
[1] SEVERITY INDICATOR + TITLE
[2] Key metrics/impact
[3] Affected entities
[4] Action button/link
```

Severity Visual Guide

<code>event.severity</code>	Slack	Teams	Email
1 (Critical)	<code>:red_circle:</code>	Red header	Red banner
2 (High)	<code>:large_orange_circle:</code>	Orange header	Orange banner
3 (Medium)	<code>:large_yellow_circle:</code>	Yellow header	Yellow banner
4 (Low)	<code>:large_blue_circle:</code>	Blue header	Blue banner

The problem record carries severity as `event.severity` on a 1–5 scale (`experimental` in the semantic dictionary), not as `CRITICAL` / `HIGH` strings. The templates below convert it with `| int(5)` , so a missing severity renders as the lowest tier; from SaaS 1.348 severity is no longer defaulted (WFLOW-04 §3). WFLOW-02 §7 maps the other old field names.

2. Jinja2 Expression Deep Dive

Variable Access

```
{{ event()["event.name"] }}      {# Direct field access #}
{{ event().get("field", "default") }} {# With default value #}
{{ result("task_name").output }} {# Previous task result #}
{{ environment().url }}          {# Environment URL #}
{{ problem_link() }}             {# Link to the problem (Problem trigger
only) #}
{{ now() }}                      {# Current timestamp #}
```

Filters

```
{{ event()["event.name"] | upper }}      {# UPPERCASE #}
{{ event()["event.name"] | lower }}       {# lowercase #}
{{ event()["event.name"] | truncate(50) }} {# Limit length #}
{{ list_field | join(", ") }}            {# Join array #}
{{ number | round(2) }}                  {# Round decimals #}
{{ timestamp | format_datetime }}        {# Format date #}
```

Conditionals

```
{% set sev = event().get("event.severity") | int(5) %}
{% if sev <= 1 %}
    :rotating_light: CRITICAL ALERT
{% elif sev == 2 %}
    :warning: HIGH ALERT
{% else %}
    :information_source: {{ event()["event.category"] }} ALERT
{% endif %}
```

Loops

```
Affected Entities:
{% for entity in event()["affected_entity_ids"][:5] %}
  - {{ entity }}
{% endfor %}
{% if event()["affected_entity_ids"] | length > 5 %}
  ... and {{ event()["affected_entity_ids"] | length - 5 }} more
{% endif %}
```

Inline Conditionals

```
{{ ":red_circle:" if (event().get("event.severity") | int(5)) <= 1 else
":large_yellow_circle:" }}

{{ event().get("root_cause_entity_id", "Pending analysis") }}
```

Dictionary Mapping

```
{# Severity (1-5) to emoji mapping #}
{{ {
  1: ":red_circle:",
  2: ":large_orange_circle:",
  3: ":large_yellow_circle:",
  4: ":large_blue_circle:"
}.get(event().get("event.severity") | int(5), ":white_circle:") }}
```

3. Slack Block Kit Templates

Full-Featured Alert Template

```
input:
  connection: slack-production
  channel: "#alerts-production"
  blocks:
    # Header with severity
    - type: header
      text:
        type: plain_text
        text: "{{ {1: ':rotating_light:', 2: ':warning:', 3:
':large_yellow_circle:', 4:
':information_source:'.get(event().get('event.severity') | int(5),
':grey_question:') }} {{ event()['event.category'] }} Problem"

    # Problem title
    - type: section
      text:
        type: mrkdown
        text: "*{{ event()['event.name'] }}"

    # Details in two columns
    - type: section
      fields:
        - type: mrkdown
          text: "*Problem ID:*{{ event()['display_id'] }}"
        - type: mrkdown
          text: "*Status:*{{ event()['event.status'] }}"
        - type: mrkdown
          text: "*Started:*{{ event()['event.start'] }}"
        - type: mrkdown
          text: "*Category:*{{ event()['event.category'] }}"

    # Root cause (if available)
    - type: section
      text:
        type: mrkdown
        text: "*Root Cause:*{{ event().get('root_cause_entity_id', 'Analysis
in progress...') }}"

    # Affected entities
    - type: section
      text:
```

```
    type: mrkdwn
    text: "*Affected ({{ event()['affected_entity_ids'] | length }}):*\n{{
event()['affected_entity_ids'][:3] | join('\n') }}{{ '\n...' if event()
['affected_entity_ids'] | length > 3 else '' }}"

# Divider
- type: divider

# Action buttons
- type: actions
  elements:
    - type: button
      text:
        type: plain_text
        text: "View Problem"
      url: "{{ problem_link() }}"
      style: primary
    - type: button
      text:
        type: plain_text
        text: "View Service"
      url: "{{ environment().url }}/ui/services"

# Footer context
- type: context
  elements:
    - type: mrkdwn
      text: "Detected by Dynatrace Dynatrace Intelligence | {{ now() }}"
```

4. Teams Adaptive Card Templates

Full-Featured Alert Card

```
input:
  connection: teams-production
  card:
    type: AdaptiveCard
    $schema: "https://adaptivecards.microsoft.com/schemas/adaptive-card.json"
    version: "1.4"
    body:
      # Header with color
      - type: TextBlock
        text: "{{ event()['event.category'] }}" Problem Detected"
        size: Large
        weight: Bolder
        color: "{{ {1: 'Attention', 2: 'Warning', 3: 'Accent', 4:
'Good'}.get(event().get('event.severity') | int(5), 'Default') }}"

      # Problem title
      - type: TextBlock
        text: "{{ event()['event.name'] }}"
        wrap: true
        weight: Bolder

      # Details table
      - type: FactSet
        facts:
          - title: "Problem ID"
            value: "{{ event()['display_id'] }}"
          - title: "Category"
            value: "{{ event()['event.category'] }}"
          - title: "Status"
            value: "{{ event()['event.status'] }}"
          - title: "Started"
            value: "{{ event()['event.start'] }}"
          - title: "Root Cause"
            value: "{{ event().get('root_cause_entity_id', 'Analyzing...') }}"

      # Affected entities
      - type: TextBlock
        text: "Affected Entities ({{ event()['affected_entity_ids'] | length
}})"
        weight: Bolder
        spacing: Medium
```

```
- type: TextBlock
  text: "{{ event()['affected_entity_ids'][:5] | join(', ') }}{{ ' ', ...'
if event()['affected_entity_ids'] | length > 5 else '' }}"
  wrap: true
  size: Small

actions:
- type: Action.OpenUrl
  title: "View in Dynatrace"
  url: "{{ problem_link() }}"
```

5. Data Enrichment

Enrich with DQL Query

Add recent error logs to notification:

```

import { execution } from '@dynatrace-sdk/automation-utils';
import { queryExecutionClient } from '@dynatrace-sdk/client-query';

async function runQuery(query) {
  // queryExecute returns a requestToken instead of the result when the query
  // outlasts
  // requestTimeoutMilliseconds - poll until it reaches a final state (see
  WFLOW-08 §2).
  const started = await queryExecutionClient.queryExecute({ body: { query,
    requestTimeoutMilliseconds: 30000 } });
  let r = started;
  while (r &&& (r.state === 'NOT_STARTED' || r.state === 'RUNNING')) {
    r = await queryExecutionClient.queryPoll({ requestToken:
      started.requestToken, requestTimeoutMilliseconds: 30000 });
  }
  if (!r || r.state !== 'SUCCEEDED') throw new Error(`DQL query did not
    succeed: ${r?.state ?? 'no response'}`);
  return r.result.records;
}

export default async function () {
  const event = (await execution()).params.event; // trigger payload
  // Get recent error logs for the affected service
  const rootCause = event.root_cause_entity_id;

  if (!rootCause) {
    return { event, recent_errors: [] };
  }

  const recentErrors = await runQuery(`
    fetch logs, from: now() - 30m
    | filter dt.entity.service == "${rootCause}"
    | filter status == "ERROR"
    | fields timestamp, content
    | sort timestamp desc
    | limit 5
  `);

  return {
    event,
    recent_errors: recentErrors
  };
}

```

Use Enriched Data in Template

```
*Recent Error Logs:*
{% for log in result("enrich_data").recent_errors[:3] %}
  • `{{ log.content | truncate(100) }}`
{% endfor %}
{% if result("enrich_data").recent_errors | length == 0 %}
  _No recent errors found_
{% endif %}
```

Enrich with Entity Details

```
import { execution } from '@dynatrace-sdk/automation-utils';
import { entitiesClient } from '@dynatrace-sdk/client-classic-environment-v2';

export default async function () {
  const event = (await execution()).params.event;
  const entityId = event.root_cause_entity_id;

  if (!entityId) {
    return { entity_name: 'Unknown', entity_type: 'Unknown' };
  }

  const entity = await entitiesClient.getEntity({
    entityId: entityId
  });

  return {
    entity_name: entity.displayName,
    entity_type: entity.type,
    tags: entity.tags || []
  };
}
```

6. Template Library

Compact Alert (Slack)

```
message: |
  {{ {1: ":red_circle:", 2: ":large_orange_circle:", 3:
":large_yellow_circle:", 4:
":large_blue_circle:"}.get(event().get("event.severity") | int(5),
":white_circle:") }} *{{ event()["event.name"] }}*
  `{{ event()["display_id"] }}` | {{ event()["event.category"] }} | &lt;{{
problem_link() }}|View&gt;
```

Problem Resolved (Slack)

```
message: |
  :white_check_mark: *Problem Resolved*

  *{{ event()["event.name"] }}*

  • *Duration:* {{ ((event().get("resolved_problem_duration") | int(0)) /
60000000000) | round(1) }} min
  • *Problem ID:* {{ event()["display_id"] }}
  • *Resolved:* {{ event().get("event.end", now()) }}
```

Daily Summary (Scheduled)

```
message: |
  :bar_chart: *Daily Problem Summary*

  *Last 24 Hours:*
  • Problems opened: {{ result("query_stats").opened }}
  • Problems resolved: {{ result("query_stats").resolved }}
  • Currently open: {{ result("query_stats").open }}

  *By Severity:*
  :red_circle: Critical: {{ result("query_stats").critical }}
  :large_orange_circle: High: {{ result("query_stats").high }}
  :large_yellow_circle: Medium: {{ result("query_stats").medium }}
```

Escalation Notice

```
message: |
  :rotating_light: *ESCALATION*

  Problem *{{ event()["display_id"] }}* has not been acknowledged after 15
  minutes.

  *{{ event()["event.name"] }}*

  This alert is being escalated to the on-call team.
  &lt;{{ problem_link() }}|View Problem&gt;
```

7. Testing Templates

Test with On-Demand Trigger

1. Create workflow with On-Demand trigger
2. Add JavaScript task to create mock event (`problem_link()` only evaluates under a Problem trigger, so link rendering cannot be tested this way):

```
export default async function() {
  return {
    // Field names match the dt.davis.problems record the Problem trigger
    delivers
    mock_event: {
      "display_id": "P-TEST-001",
      "event.id": "-1234567890123456789_17900000000000V2",
      "event.name": "High response time on checkout-service",
      "event.category": "SLOWDOWN",
      "event.severity": "1",
      "event.status": "ACTIVE",
      "event.start": new Date().toISOString(),
      "affected_entity_ids": ["SERVICE-ABC123", "SERVICE-DEF456"],
      "root_cause_entity_id": "HOST-XYZ789"
    }
  };
}
```

3. Use mock data in notification:

```
message: |
  {{ result("mock_data").mock_event["event.category"] }} Alert
  {{ result("mock_data").mock_event["event.name"] }}
```

Preview Templates in Slack Block Kit Builder

Use <https://app.slack.com/block-kit-builder> to preview your block templates before deployment.

Query Notification Effectiveness

```
// Notification task performance
// Data object corrected 08/12/2026. Workflow executions are NOT in `events`,
// and there is no
// `automation.task.execution` / `automation.workflow.execution` event type in
// any spelling – those
// filters matched nothing, silently, in 25 cells. AutomationEngine writes to
// `dt.system.events` with
// `event.kind == "WORKFLOW_EVENT"` (5.7M records / 30d here), split by
// `event.type`:
//   WORKFLOW_EXECUTION · TASK_EXECUTION · ACTION_EXECUTION ·
//   WORKFLOW_CREATED/UPDATED/DELETED
// Fields are the `dt.automation_engine.*` family:
//   .workflow.title / .workflow.id      (was workflow.name)
//   .task.name                          (was task.name)
//   .action.function / .action.app      (was task.type – e.g. run-javascript,
//   send-email,
//                                       snow-search-incidents)
//   .state                             (was task.status / execution.status)
//                                       values RUNNING · SUCCESS · ERROR ·
//   DISCARDED · SKIPPED
//                                       – note "FAILED" is NOT a value; it
//   is ERROR
//   .state.is_final                    true only on terminal records –
//   filter on it, otherwise a
//                                       single execution is counted once as
//   RUNNING and again as
//                                       SUCCESS/ERROR
//   .state_info                        (was task.error) · duration (was
//   task.duration)
// Enumerate with:
//   fetch dt.system.events, from:-24h | filter event.kind == "WORKFLOW_EVENT"
//   | limit 1
//   fetch dt.system.events, from:-7d
//   | filter event.kind == "WORKFLOW_EVENT"
//   | filter event.type == "ACTION_EXECUTION"
//   | filter dt.automation_engine.state.is_final == true
//   | summarize {executions = count(), avg_ms = avg(duration) / 1ms, p95_ms =
//   percentile(duration, 95) / 1ms}, by:{dt.automation_engine.action.function}
//   | sort p95_ms desc
//   | limit 20
```

```

// Task errors (malformed templates surface here)
// Data object corrected 08/12/2026. Workflow executions are NOT in `events`,
and there is no
// `automation.task.execution` / `automation.workflow.execution` event type in
any spelling – those
// filters matched nothing, silently, in 25 cells. AutomationEngine writes to
`dt.system.events` with
// `event.kind == "WORKFLOW_EVENT"` (5.7M records / 30d here), split by
`event.type`:
//   WORKFLOW_EXECUTION · TASK_EXECUTION · ACTION_EXECUTION ·
WORKFLOW_CREATED/UPDATED/DELETED
// Fields are the `dt.automation_engine.*` family:
//   .workflow.title / .workflow.id      (was workflow.name)
//   .task.name                          (was task.name)
//   .action.function / .action.app      (was task.type – e.g. run-javascript,
send-email,
//                                       snow-search-incidents)
//   .state                             (was task.status / execution.status)
//                                       values RUNNING · SUCCESS · ERROR ·
DISCARDED · SKIPPED
//                                       – note "FAILED" is NOT a value; it
is ERROR
//   .state.is_final                    true only on terminal records –
filter on it, otherwise a
//                                       single execution is counted once as
RUNNING and again as
//                                       SUCCESS/ERROR
//   .state_info                        (was task.error) · duration (was
task.duration)
// Enumerate with:
//   fetch dt.system.events, from:-24h | filter event.kind == "WORKFLOW_EVENT"
| limit 1
fetch dt.system.events, from:-24h
| filter event.kind == "WORKFLOW_EVENT"
| filter event.type == "ACTION_EXECUTION"
| filter dt.automation_engine.state == "ERROR"
| filter isNotNull(dt.automation_engine.state_info) and
dt.automation_engine.state_info != ""
| summarize errors = count(), by:{dt.automation_engine.action.function,
dt.automation_engine.state_info}
| sort errors desc
| limit 25

```

Next Steps

With custom templates ready, implement auto-remediation:

Recommended Path

1. **WFLOW-07: Problem-Triggered Remediation** - Auto-remediation patterns
2. **WFLOW-08: JavaScript & HTTP Actions** - Custom integrations
3. **WFLOW-09: Security & Governance** - Best practices

Key Takeaways

- **Jinja2** enables dynamic, conditional content
 - **Slack Block Kit** creates rich, interactive messages
 - **Teams Adaptive Cards** provide structured formatting
 - **Data enrichment** adds context from DQL queries
 - **Test templates** with mock data before production
-

Summary

In this notebook, you learned:

- Alert message design principles
 - Jinja2 expressions, filters, and conditionals
 - Slack Block Kit template structure
 - Teams Adaptive Card formatting
 - Data enrichment with DQL and entity lookups
 - Template library patterns
 - Testing strategies
-

References

- [Workflow reference / Jinja expressions \(DT docs\)](#)
 - [Notification actions umbrella \(DT docs\)](#)
 - [Jinja2 template designer \(Pallets\)](#)
 - [Slack Block Kit reference \(Slack API\)](#)
 - [Slack Block Kit Builder \(Slack\)](#)
 - [Teams Adaptive Cards \(Microsoft Learn\)](#)
-

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.